
ENGINEERING HANDBOOK · 2026

macOS 开发者工作站手册 从新机上手到可复现工程环境

系统快捷操作、软件分层、Homebrew 与 Brewfile、终端、运行时、容器、权限、备份、维护 SOP 和独立 Cheatsheet。

{Winston}版本 V1.0 · 2026-06-02

资料口径：macOS Tahoe 26.5 与公开资料，截至 2026-06-02

| 目录

01	执行摘要	03
02	1. 先建立工作站心智模型	04
03	2. 新机开箱前 15 分钟	06
04	3. 60 分钟落地路径	08
05	4. macOS 快捷操作: 先掌握高频路径	12
06	5. 触控板、窗口与桌面	15
07	6. 软件推荐矩阵: 默认、按需、谨慎	17
08	7. Homebrew 与 Brewfile: 把新机安装变成契约	19
09	8. Ghostty 与 shell: 少配一点	21
10	9. mise 与 uv: 让项目环境回到项目	22
11	10. OrbStack: 容器、Linux 与本地 Kubernetes	25
12	11. Mole: 维护先观察, 删除要预演	27
13	12. 日常 SOP: 每天、每周、每月	28
14	13. 权限、安全与隐私	30
15	14. 社区推荐如何转化为可靠方案	32
16	15. 三种工作站配置	34
17	16. 排障手册	36
18	17. 最佳食用方式	38
19	18. 发布前自检	39
20	附录: macOS Developer Workstation Cheatsheet	40
21	附录: 资料来源与边界	49

执行摘要

一台真正好用的 Mac，不是安装了最多软件的 Mac，而是摩擦足够少、行为足够稳定、出了问题能够恢复的 Mac。

这份手册给出一套保守但高效的开发者工作站方案。核心思路只有五条：

1. 先学会原生能力，再引入增强工具。Spotlight、Finder、Quick Look、原生窗口平铺、触控板和系统快捷键已经能覆盖大量高频动作。
2. 把安装清单写成契约。Homebrew 负责工作站级应用和命令行工具，Brewfile 进入版本控制。新机不再依赖记忆。
3. 把项目运行时放回项目。mise 负责 Node、Python、Java 等运行时和任务入口，uv 负责 Python 项目依赖。全局环境只保留薄薄一层。
4. 有容器需求再安装容器工具。OrbStack 适合在 macOS 上运行 Docker、Linux machines 和本地 Kubernetes，但商业使用要先确认许可。
5. 清理、权限和升级都要有刹车。Mole 默认先用 `mo status` 和 `mo analyze`；删除动作先 `--dry-run`。权限按需授予，备份先于优化。

本手册没有把所有口碑软件都列为“必装”。Linux.do 的长期讨论说明，真正的痛点集中在窗口管理、启动器、剪贴板、菜单栏、压缩、媒体播放、跨平台传输、外接显示器、容器和多语言环境。对每类痛点，先给原生方案，再给一个受控的增强路径。

建议把本文当成三份文档使用：

- 第一次拿到新机：按第 2 章和第 3 章执行。
- 日常维护：按第 12 章和第 13 章执行。
- 临时查找：直接打印附录 Cheatsheet。



Figure 1. 把工作站拆成四层：机器只维护工作站入口，项目自己携带运行时和依赖契约。

1. 先建立工作站心智模型

macOS 的优势不是某一个单点功能，而是系统动作、应用动作、文件动作和 Unix 工具链能够连成一条较短的路径。好的配置应当减少上下文切换，而不是增加更多后台常驻进程。

1.1 四层职责

层级	负责什么	典型工具	不负责什么
系统层	更新、加密、权限、备份、窗口、输入	macOS 原生设置	项目依赖
工作站层	通用 CLI、桌面应用、容器入口	Homebrew、Brewfile、Ghostty、OrbStack	每个项目的运行时版本
项目层	运行时、环境变量、任务	mise、mise.toml 、 mise.lock	全局安装所有语言工具
语言层	项目依赖与锁文件	uv、npm、pnpm、Cargo 等	修改系统 Python

当职责混在一起时，新机会变得不可恢复：Node 装在 Homebrew，另一个版本装在 nvm，Python 混用系统解释器、Conda 和随机 `pip install`，Java 散落在多个目录，Docker 与本机依赖又各自维护一份说明。短期能运行，长期难以解释。

更好的方向是让每一层都足够薄：

- Homebrew 安装少量稳定的工作站级工具。
- Brewfile 记录“这台机器需要什么”。
- mise 记录“这个项目需要什么版本”。
- uv 或其他语言包管理器记录“这个项目依赖什么库”。
- 容器记录“哪些服务应该隔离运行”。

1.2 原生优先不是拒绝第三方

原生优先的价值在于建立基线。只有知道原生方案的上限，才能判断某个第三方工具是否真的减少摩擦。

需求	先用什么	什么时候升级
搜索与打开应用	Spotlight	需要动作、Quicklinks、Snippets、扩展时再用 Raycast
窗口排列	原生平铺	需要键盘驱动布局或自动平铺时再选 Loop、Rectangle、AeroSpace
文件预览	Finder + Space	原生无法打开特定格式时再补工具
文件传输	AirDrop	有 Windows、Linux、Android 设备时再用 LocalSend
状态观察	活动监视器	需要长期菜单栏观察时再用 Stats

第三方工具应解决明确痛点。不能回答“它替代了哪一步”的工具，不必进入基线。

| 2. 新机开箱前 15 分钟

不要从安装软件开始。先完成系统更新、磁盘加密、备份和权限基线。这些动作不显眼，但决定了后续升级、迁移和事故恢复的成本。

2.1 确认系统版本

打开 Apple 菜单 > 关于本机，确认 macOS 名称、版本和硬件信息。Apple 在 2026-05-11 更新的版本页中列出 macOS Tahoe 26.5 为 Tahoe 最新版本；在 2026-06-02 新建工作站时，应先通过系统设置 > 通用 > 软件更新 检查实际可用更新。

Homebrew 官方安装文档当前将 macOS Sonoma 14 或更高版本列为受支持基线。旧系统可能仍能运行部分软件，但不要把“可能工作”当成团队标准。

2.2 开启 FileVault

路径：系统设置 > 隐私与安全性 > FileVault。

FileVault 用于加密启动磁盘。开启前先确定恢复密钥的保存方式：个人设备可保存在可靠的密码管理器中；公司设备遵循组织 MDM 和密钥托管规则。不要把恢复密钥只放在这台 Mac 上。

验收问题：

- FileVault 是否已开启？
- 恢复密钥是否存在于设备之外？
- 公司设备是否满足组织托管要求？

2.3 建立 Time Machine

路径：系统设置 > 通用 > Time Machine。

Time Machine 是迁移和恢复的第一道保险。它不能替代 Git、云同步和项目级备份，但能显著降低系统升级、磁盘故障和误删后的恢复成本。

首次备份完成后，至少确认一次：

1. 菜单栏或系统设置能看到最近备份时间。
2. 能进入 Time Machine 浏览历史文件。
3. 外接盘或网络目标有足够空间。
4. 恢复目标不是唯一一份重要数据。

2.4 权限从最小集开始

路径：系统设置 > 隐私与安全性。

重点观察：

- 辅助功能

- 屏幕与系统音频录制
- 完全磁盘访问权限
- 文件与文件夹
- 输入监控
- 自动化
- 登录项与扩展

窗口管理器、剪贴板工具、截图工具、终端和自动化工具都可能请求额外权限。原则是：先理解用途，再授予；不再使用，立即回收。

2.5 先别执行这些动作

- 不运行来源不明的一键初始化脚本。
- 不粘贴大段未经解释的 `defaults write`。
- 不关闭系统安全机制来换取几秒便利。
- 不急着安装多个同类工具。
- 不在完成备份前运行深度清理。

| 3. 60 分钟落地路径

这一章是一条最短可用路径。完成后，你会得到一台可开发、可复查、可迁移的 Mac。没有容器需求的用户可以跳过 OrbStack。

3.1 安装 Command Line Tools

```
xcode-select --install
xcode-select -p
```

Command Line Tools 提供编译器、Git 和基础开发工具。Homebrew 官方文档将其列为 macOS 安装要求之一。

3.2 安装 Homebrew

从 [brew.sh](#) 阅读并复制官方安装命令。不要从博客、论坛回帖或随机脚本镜像安装。

Homebrew 官方文档说明：

- Apple Silicon 默认前缀为 `/opt/homebrew`。
- Intel Mac 默认前缀为 `/usr/local`。
- 默认前缀可以使用多数预编译 bottles。
- 初次安装后，日常 `brew install` 通常不需要 `sudo`。

Apple Silicon 常见 shell 初始化：

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```

如果安装器输出了不同提示，以安装器为准。不要机械复制别人的 PATH。

验收：

```
brew --version
brew doctor
```

3.3 用 Brewfile 安装基础工具

在一个受版本控制的工作站配置仓库中创建 Brewfile：


```
brew "bat"
brew "eza"
brew "fd"
brew "fzf"
brew "gh"
brew "git-delta"
brew "jq"
brew "mise"
brew "mole"
brew "ripgrep"
brew "uv"
brew "zoxide"

cask "ghostty"
# cask "orbstack" # 有容器或 Linux 需求时启用
# cask "raycast" # 原生 Spotlight 不够用时启用
```

应用契约：

```
brew bundle check --file=./Brewfile || brew bundle install --file=./Brewfile
```

这里故意没有塞入几十个软件。基线越小，越容易解释、迁移和升级。

3.4 配置 Git 与 GitHub CLI

```
git config --global --edit
git config --global init.defaultBranch main
git config --global pull.ff only
git config --global fetch.prune true

gh auth login
gh auth status
```

在编辑器中填写真实的 `user.name` 和 `user.email`。不要把示例身份复制进自己的配置。

如果团队使用 SSH，再验证：

```
ssh -T git@github.com
```

3.5 配置最小 zsh

不要在第一小时安装大型 shell 框架。先保留一个能解释的 `.zshrc`：

```
if [[ -x /opt/homebrew/bin/brew ]]; then
    eval "$(/opt/homebrew/bin/brew shellenv)"
fi

eval "$(mise activate zsh)"
eval "$(zoxide init zsh)"

alias ll='eza -lah --group-directories-first'
alias cat='bat --paging=never'
```

关闭并重新打开终端，然后验收：

```
which brew
which mise
which uv
which rg
mise doctor
```

3.6 有容器需求时安装 OrbStack

```
brew install orbstack
docker context show
docker run --rm hello-world
```

OrbStack 官方 quick start 说明它包含 Docker engine、Compose 和 Linux machines。默认即可工作，不要先进入复杂配置。

商业、非营利组织、政府实体或专业自由职业用途需要按官方 licensing 页面确认许可。团队推广前由负责人统一处理。

3.7 一小时验收清单

检查	命令或路径	通过标准
系统	系统设置 > 通用 > 软件更新	无待处理关键更新
加密	系统设置 > 隐私与安全性 > FileVault	已开启并保存恢复方式
备份	系统设置 > 通用 > Time Machine	能看到最近备份
Homebrew	brew doctor	无需立即处理的关键错误
工具链	brew bundle check --file=./Brewfile	契约满足
GitHub	gh auth status	已登录正确账户
mise	mise doctor	环境无明显错误
uv	uv --version	命令可用
Docker	docker run --rm hello-world	仅容器用户需要通过

4. macOS 快捷操作：先掌握高频路径

macOS 的快捷键不是 Windows 快捷键的一对一翻译。最重要的变化是：Command 承担大部分应用级动作，Option 经常表示替代动作或细粒度动作，Control 更多参与系统级动作和终端动作。

4.1 五个修饰键

符号	名称	建议理解
⌘	Command	应用主修饰键，复制、保存、切换、退出
⌥	Option / Alt	替代动作、按词移动、打开额外设置
⌞	Control	Mission Control、锁屏、终端组合键
⇧	Shift	反向、范围和组合
fn / Globe	Function / Globe	功能键、输入源和系统入口

4.2 应用与窗口不是一回事

macOS 中关闭窗口通常不会退出应用。

快捷键	动作
⌘ Tab	切换应用
⌘ `	切换当前应用内的窗口
⌘ W	关闭当前窗口或标签
⌘ Q	退出当前应用
⌘ H	隐藏当前应用
⌥ ⌘ Esc	强制退出应用
⌘ ,	打开当前应用设置

这是从其他桌面系统迁移时最值得先建立的肌肉记忆。

4.3 Spotlight 是默认入口

使用 ⌘ Space 打开 Spotlight。先把它用于：

- 搜索并打开应用
- 查找文件
- 简单计算
- 单位换算

- 查找系统设置
- 执行 Spotlight 支持的动作

Spotlight 足够时，不需要立刻安装启动器。只有当你反复遇到以下需求时，Raycast 才值得进入工作站：

- 用快捷键执行窗口动作
- 维护 Quicklinks
- 维护 Snippets
- 需要扩展生态
- 需要受控的剪贴板历史

4.4 Finder 是文件操作中枢

开发者常把 Finder 当成“图形化文件浏览器”，但它的 Quick Look、路径跳转和文件移动动作很适合高频使用。

快捷键	动作
Space	Quick Look 预览
⇧ ⌘ G	前往文件夹
⇧ ⌘ H	用户目录
⇧ ⌘ L	下载目录
⇧ ⌘ U	实用工具目录
⇧ ⌘ P	显示或隐藏路径栏
⇧ ⌘ V	将复制的文件移动到当前位置
⌘ ↑	上级目录
⌘ Delete	移到废纸篓

建议打开路径栏。它能减少在 Finder 与终端之间来回确认目录的成本。

4.5 截图、录屏和演示

快捷键	动作
⇧ ⌘ 3	截取全屏
⇧ ⌘ 4	截取选区
⇧ ⌘ 5	截图与录屏面板

需要屏幕录制的應用可能请求 屏幕与系统音频录制 权限。完成临时任务后，应回收不再需要的权限。

4.6 文本编辑是效率放大器

快捷键	动作
<code>⌘ ← / ⌘ →</code>	按词移动
<code>⌘ ← / ⌘ →</code>	行首 / 行尾
<code>⌘ ↑ / ⌘ ↓</code>	文首 / 文末
<code>⌘ Delete</code>	删除左侧一个词
<code>fn Delete</code>	向右删除
<code>⇧ 加移动键</code>	扩展选择范围

这些动作在编辑器、浏览器输入框、聊天工具和系统表单中普遍可用，比堆叠更多编辑器插件更先产生收益。

5. 触控板、窗口与桌面

鼠标和键盘不是唯一入口。macOS 的触控板、Mission Control、Hot Corners 和原生窗口平铺能够形成一套低负担桌面工作流。

5.1 先调触控板

路径：系统设置 > 触控板。

建议逐项试用：

- 轻点来点按
- 双指滚动
- 双指辅助点按
- 三指或四指上推进入 Mission Control
- 三指或四指左右滑动切换桌面
- 放大、旋转和智能缩放

拖移窗口时，可按 Apple 文档在辅助功能设置中启用三指拖移。是否启用取决于个人习惯，不需要强求统一。

5.2 Mission Control 与桌面

快捷键	动作
<code>^ ↑</code>	Mission Control
<code>^ ↓</code>	当前应用的所有窗口
<code>^ ← / ^ →</code>	切换桌面

桌面不宜过多。通常按工作上下文分为三到五个即可，例如沟通、编辑器、浏览器、终端和临时演示。

5.3 原生窗口平铺优先

在较新的 macOS 中，先通过 系统设置 > 桌面与程序坞 > 窗口 查看原生平铺设置。原生方案适合：

- 左右二分
- 将窗口拖到屏幕边缘
- 使用系统提供的平铺动作
- 低权限、低维护的基础布局

只有当你明确需要更多键盘动作、自动布局或复杂多显示器规则时，再安装一个增强工具。

5.4 窗口增强工具只选一个

工具	适合什么人	代价
Loop	喜欢直观布局与快捷操作	多一个常驻工具与权限面
Rectangle	想要传统、轻量的键盘窗口动作	仍需记忆一套快捷键
AeroSpace	需要自动平铺和更强键盘驱动	学习成本更高，不适合作为默认

同时安装多个窗口管理器只会制造快捷键冲突和排障成本。

5.5 Hot Corners 慎用

Hot Corners 适合屏幕保护、锁屏、桌面等低风险动作。不要把高频误触会打断工作的动作放在最常经过的角落。配置完成后至少使用一天，再决定是否保留。

6. 软件推荐矩阵：默认、按需、谨慎

“必装软件”容易演变为不受控制的购物清单。更稳妥的做法是按职责分层。

6.1 默认安装

工具	作用	为什么进入默认层
Homebrew	工作站包管理	建立统一安装入口
Brewfile	安装契约	支持复查、迁移和差异对比
gh	GitHub CLI	登录、仓库和协作入口
rg、fd、fzf、jq	查找与处理数据	高频、轻量、可组合
bat、eza、zoxide	更顺手的浏览与跳转	增强终端但不改变系统职责
mise	项目运行时与任务	避免运行时散落
uv	Python 项目和工具	避免项目依赖污染全局
Mole	状态观察与受控维护	先读后删，保留安全边界

Ghostty 可以与默认层一起安装，但 Terminal.app 本身已是可靠兜底。

6.2 有明确痛点再安装

工具	安装条件	注意事项
Ghostty	终端使用频繁，希望更现代的原生体验	先零配置使用
OrbStack	有 Docker、Linux 或本地 Kubernetes 需求	商业许可先确认
Raycast	Spotlight 不足以覆盖动作、Quicklinks、Snippets	复核权限和扩展来源
Ice	菜单栏拥挤	先减少不必要常驻项
Stats	需要长期观察 CPU、内存、磁盘、网络	Sensors、Bluetooth 模块可能提高资源消耗
IINA	需要更强媒体播放	从官方渠道获取
Keka	需要更多压缩格式	从官网、App Store 或 Homebrew 获取
LocalSend	经常与非 Apple 设备传文件	适合局域网跨平台传输
BetterDisplay	外接显示器存在缩放、亮度等问题	问题存在时再装
Maccy	需要独立剪贴板历史且未使用 Raycast Clipboard	复核敏感内容边界
Pearcleaner	偏好 GUI 卸载和残留查看	与 Mole 不必同时作为默认
Applite	希望图形化安装 Homebrew cask	开发者基线仍以 Brewfile 为准
chezmoi	已有稳定 dotfiles，准备跨机器维护	不要在首日过度抽象
mas	需要记录 App Store 应用	只能覆盖 App Store 范围

6.3 谨慎处理

类别	风险	建议
代理、VPN、网络过滤	地域、组织合规、证书和流量边界复杂	按组织策略单独评审
电池控制工具	习惯、硬件和收益因人而异	先用系统优化充电
商业清理工具	可能与系统能力、Mole、Pearcleaner 重叠	不设为统一基线
一键安装器	容易隐藏脚本、权限和来源	只在逐项审阅后使用
非官方安装包	完整性、更新和许可不可控	不进入工作站

7. Homebrew 与 Brewfile: 把新机安装变成契约

Homebrew 的价值不只是“能安装软件”，而是让机器状态有一个可检查的入口。

7.1 工作站级工具才进入 Brewfile

适合写入 Brewfile:

- 高频通用 CLI
- 桌面应用
- 团队明确统一的基础工具
- 容器入口

不适合写入 Brewfile:

- 单个项目的 Node、Python、Java 版本
- Python 项目的库依赖
- 临时实验工具
- 未验证来源的 tap

7.2 最小 Brewfile

```
brew "bat"
brew "eza"
brew "fd"
brew "fzf"
brew "gh"
brew "git-delta"
brew "jq"
brew "mise"
brew "mole"
brew "ripgrep"
brew "uv"
brew "zoxide"

cask "ghostty"
```

按需附加:

```
cask "orbstack"
cask "raycast"
cask "jordanbaird-ice"
cask "stats"
cask "iina"
cask "keka"
cask "localsend"
```

这不是让所有人取消注释。它是候选清单，提交前应审阅。

7.3 日常命令

```
brew search <name>
brew info <name>
brew list
brew outdated

brew bundle check --file=./Brewfile
brew bundle install --file=./Brewfile

brew update
brew upgrade
brew cleanup
brew doctor
```

7.4 导出与清理要分开

Homebrew 官方文档提供：

```
brew bundle dump --global --force --describe
```

它适合导出现状，但导出后必须审阅 diff。机器上临时安装过的软件，不一定值得进入长期契约。

更需要谨慎的是：

```
brew bundle cleanup --file=./Brewfile
```

先运行不带 `--force` 的预览，阅读要移除的内容。不要在不了解差异时执行强制清理。

7.5 Analytics 是可见的选择

Homebrew 官方文档说明其匿名 analytics 会在首次相关操作时提示，用户可关闭：

```
brew analytics off
```

这不是必须执行的初始化命令。团队应根据隐私政策做明确选择，而不是默默改变用户设置。

| 8. Ghostty 与 shell: 少配一点

终端是开发工作站的入口, 但终端配置不应成为一项长期维护工程。

8.1 为什么推荐 Ghostty

Ghostty 官方文档强调零配置理念: 应用开箱即用, 内置默认字体 JetBrains Mono 和常见能力。macOS 官方分发的 `.dmg` 经过签名与 notarization; Homebrew cask 重新打包官方 `.dmg`。

安装:

```
brew install --cask ghostty
```

第一周建议不写配置文件。只有遇到明确偏好时再调整。

8.2 配置文件位置

Ghostty 官方文档列出 XDG 和 macOS 专用路径。建议使用 XDG 路径, 便于 dotfiles 管理:

```
~/.config/ghostty/config.ghostty
```

最小示例:

```
# 只保留确实需要的主观偏好
font-family = JetBrains Mono
```

macOS 默认可用 `cmd+shift+,` 重新加载配置。配置项较多时, 先通过官方 option reference 核验, 不要复制多年累积的配置包。

8.3 shell 只保留可解释入口

推荐 `.zshrc`:

```
if [[ -x /opt/homebrew/bin/brew ]]; then
  eval "$(/opt/homebrew/bin/brew shellenv)"
fi

eval "$(mise activate zsh)"
eval "$(zoxide init zsh)"

alias ll='eza -lah --group-directories-first'
alias cat='bat --paging=never'
```

如果 shell 启动变慢, 先诊断:

```
zsh -xlic exit
```

处理顺序是移除不必要插件、缩减同步网络调用、确认 PATH, 再考虑更复杂的缓存策略。

9. mise 与 uv: 让项目环境回到项目

Linux.do 的开发环境迁移讨论反复提到同一个痛点：JDK、Python、Node、包管理器和工具散落在不同入口，时间久了没人知道哪个版本正在生效。mise 与 uv 的组合可以把职责重新整理清楚。

9.1 分工

工具	负责什么	示例
Homebrew	安装 mise 与 uv 本身	<code>brew install mise uv</code>
mise	管理项目运行时、环境变量、任务	<code>Node、Python、Java、mise run test</code>
uv	管理 Python 项目依赖和 Python CLI 工具	<code>uv add 、 uv run 、 uv tool install</code>

9.2 mise 的最小使用

交互 shell 激活：

```
eval "$(mise activate zsh)"
```

全局只保留少量默认值：

```
mise use --global node@lts
```

项目内写入 `mise.toml`：

```
cd <project>
mise use node@lts
mise use python@3.12
mise install
```

典型 `mise.toml`：

```
[tools]
node = "lts"
python = "3.12"

[tasks.test]
run = "pytest"

[tasks.lint]
run = "ruff check ."
```

运行：

```
mise run test
mise run lint
```

9.3 陌生项目先审阅再信任

mise 会在拉取别人编写的配置后提示信任。原因很直接：配置可能包含环境变量、任务和 hooks。正确做法不是机械接受，而是：

1. 阅读 `mise.toml`。
2. 检查任务脚本和 hooks。
3. 确认来源仓库。
4. 再执行信任与安装。

9.4 锁文件用于团队复现

mise 官方文档说明 `mise.lock` 用于锁定精确版本和 checksums。锁文件不会自动首次创建，需要显式启用并生成：

```
mise settings lockfile=true
mise lock
git add mise.toml mise.lock
```

更新流程：

```
mise use node@lts
mise install
git diff -- mise.toml mise.lock
```

团队成员拉取后：

```
mise install
```

9.5 uv 的项目 workflow

安装：

```
brew install uv
```

新项目：

```
uv init hello-world
cd hello-world
uv add requests
uv run python -c 'import requests; print(requests.__version__)'
```

已有项目：

```
uv sync
uv run pytest
```

Python CLI 工具：

```
uv tool install ruff
```

uv 会把项目依赖记录在 `pyproject.toml` 和 `uv.lock`。不要习惯性把项目包安装到系统 Python 或全局环境。

10. OrbStack: 容器、Linux 与本地 Kubernetes

OrbStack 不是每台 Mac 的必装项。它适合明确需要 Docker、Linux machines 或本地 Kubernetes 的开发者。

10.1 从默认配置开始

```
brew install orbstack
docker context show
docker run --rm hello-world
```

OrbStack 会创建名为 `orbstack` 的 Docker context，并在终端中自动使用。Compose 也已包含：

```
docker compose up -d
docker compose ps
docker compose down
```

10.2 文件与 volume

容器持久数据优先使用 volume。OrbStack quick start 建议使用 `~/OrbStack/docker` 以获得合适的文件系统行为。不要把大型依赖目录无意识地放到跨边界 bind mount 中，再把性能问题归因于容器本身。

10.3 Apple Silicon 与 x86

OrbStack 官方 Docker 文档说明，在 Apple Silicon 上可使用 Rosetta 运行 Intel `x86_64/amd64` 镜像。确有兼容性需求时显式指定：

```
docker run --platform linux/amd64 <image>
```

默认仍应优先使用原生 ARM 镜像。跨架构运行是兼容路径，不是默认路径。

10.4 Linux machines

创建 Linux machine：

```
orb create ubuntu devbox
orb -m devbox uname -a
orb logs devbox
```

文件边界：

方向	路径
Linux 中访问 Mac 文件	<code>/mnt/mac</code>
Mac 中访问 Linux 文件	<code>~/OrbStack</code> 或 Finder 集成

OrbStack 默认会为 Linux machine 创建与 macOS 同名的用户，并配置 `passwordless sudo`。这适合本地开发，但不要误认为是生产服务器安全模型。

10.5 Docker TCP 不要裸露

OrbStack 官方 Docker 文档明确提醒：把 Docker TCP 暴露到 `0.0.0.0:2375` 非常危险。需要远程访问时，使用 SSH 或带客户端认证的 TLS。

10.6 本地 Kubernetes

有 Kubernetes 开发需求时再启用。OrbStack 提供轻量单节点集群、GUI、网络集成和 `kubect1`。端口默认只对 localhost 可访问；在不可信网络上不要随意开放到局域网。

10.7 商业许可

OrbStack 官方 licensing 页面说明，专业自由职业、商业、非营利组织和政府实体用途需要购买许可。个人非商业用途和非商业教育用途边界不同。团队部署前必须让负责人阅读当前许可页，而不是依赖旧印象。

11. Mole: 维护先观察, 删除要预演

社区讨论里, Mole 经常作为 macOS 清理工具出现, 也有用户反馈深度清理影响了 VPN。这个反例很重要: 清理工具不是越激进越好。

11.1 默认从只读动作开始

```
brew install mole
mo status
mo analyze
```

`mo status` 用于观察系统健康状态。`mo analyze` 用于可视化磁盘占用; 官方 README 说明, 它通过 Finder 把临时处理的文件移到废纸篓, 而不是直接删除。

11.2 删除动作先 `--dry-run`

```
mo clean --dry-run
mo uninstall --dry-run
mo purge --dry-run
```

Mole 官方 README 将 `clean`、`uninstall`、`purge`、`installer` 和 `remove` 归为破坏性操作。先预演, 再逐项阅读。

回看日志:

```
mo history
mo history --json
```

操作日志路径:

```
~/Library/Logs/mole/operations.log
```

11.3 不要自动化深度清理

不要把 `mo clean`、`mo purge` 或任何类似工具写进无人值守定时任务。开发机器里常见的模拟器、SDK、VPN、证书、缓存、构建产物和挂载卷都需要上下文判断。

更稳妥的月度流程:

1. 确认最近 Time Machine 备份。
2. 运行 `mo status`。
3. 使用 `mo analyze` 查看大目录。
4. 必要时运行对应 `--dry-run`。
5. 阅读删除列表。
6. 只处理明确知道用途的内容。

| 12. 日常 SOP: 每天、每周、每月

工作站维护不需要成为一项爱好。固定少量节奏，比偶尔进行一次大扫除更可靠。

12.1 每天：开始工作

项目目录内：

```
git status --short --branch  
mise current
```

有容器时：

```
docker compose ps
```

目的不是制造仪式，而是尽早发现错误分支、未提交改动、版本漂移和异常服务。

12.2 每周：查看变化

```
brew update  
brew outdated  
brew bundle check --file=./Brewfile  
mo status
```

每周只查看，不要求立即升级所有工具。关键项目发布期或出差前，稳定性优先。

12.3 每月：受控升级

```
brew upgrade  
brew cleanup  
brew doctor  
mo analyze
```

同时检查：

- Time Machine 最近备份时间
- 登录项
- 辅助功能、录屏、完全磁盘访问权限
- 菜单栏常驻模块
- 磁盘余量
- 不再使用的容器、镜像和 volume
- dotfiles 是否已经提交

12.4 大版本升级前

1. 完成一次 Time Machine 备份。
2. 导出并审阅 Brewfile。
3. 提交 dotfiles 与重要项目变更。
4. 记录 VPN、证书、组织策略和关键应用许可。
5. 检查容器与本地数据库是否有独立备份。
6. 暂停深度清理和批量系统修改。
7. 阅读关键开发工具的兼容性说明。

12.5 换机或重装

恢复顺序：

1. 系统更新与 FileVault。
2. Time Machine 恢复必要数据。
3. Command Line Tools。
4. Homebrew。
5. Brewfile。
6. GitHub CLI 和 SSH。
7. dotfiles。
8. mise 与项目锁文件。
9. uv 与项目依赖。
10. OrbStack、容器、Linux machines。
11. 按痛点补充 GUI 工具。

不要一开始就恢复所有旧软件。换机也是清理工作站契约的机会。

13. 权限、安全与隐私

macOS 上的便利工具经常需要系统权限。权限不是“点一下允许”这么简单，它决定了应用能观察或控制什么。

13.1 常见权限与原因

权限	常见用途	复核问题
辅助功能	窗口控制、自动粘贴、全局快捷键	仍在使用该工具吗？
屏幕与系统音频录制	截图、录屏、会议、演示	临时任务结束后还需要吗？
完全磁盘访问权限	备份、终端、索引、清理	是否有更小权限可用？
输入监控	键盘增强、自动化	来源和用途是否可信？
自动化	控制其他应用	被控制应用范围是否合理？
登录项	常驻启动	是否值得每次登录都启动？

13.2 剪贴板工具要单独评审

剪贴板历史很方便，也可能收集敏感片段。Raycast 官方文档说明其 Clipboard History 在本地加密，并对密码管理器复制内容做排除；Maccy 官方 README 也说明默认忽略一组被视为 confidential 的类型，包括 1Password 相关类型。

即便如此，仍要遵守三条规则：

1. Raycast Clipboard 与 Maccy 二选一。
2. 检查排除规则是否符合自己的密码管理器和工作应用。
3. 不在高敏感环境默认开启长期历史。

13.3 下载渠道

优先顺序：

1. App Store
2. 项目官网
3. Homebrew 官方 formula 或 cask
4. 项目官方 GitHub Releases

社区帖子可以帮你发现软件，但不要从回帖中的第三方存储链接下载旧版安装包。来源、签名、更新渠道和许可都更难确认。

13.4 不盲目执行 shell 命令

看到任何一键安装、系统优化或权限修改命令时，先回答：

- 它从哪里下载内容？
- 是否有 `sudo` ？
- 是否修改 PATH、shell profile、系统设置或安全机制？
- 是否能预演？
- 是否能回滚？
- 是否有独立备份？

无法回答时，不执行。

14. 社区推荐如何转化为可靠方案

Linux.do 的讨论很有价值，因为它呈现了真实工作流：有人需要更好的播放器和解压工具，有人迁移到 uv 与 mise，有人希望新机安装不再遗漏 Docker 和 Node，也有人遇到清理工具影响 VPN 的反例。

但社区口碑需要经过三步转化。

14.1 第一步：识别痛点

社区信号	背后痛点
Raycast 高频出现	启动、动作、窗口、快捷链接、片段
OrbStack 高频出现	macOS 容器和 Linux 开发体验
Mole 与 Pearcleaner	磁盘观察、卸载残留、维护
Keka 与 IINA	原生压缩、媒体能力不足
LocalSend	非 Apple 设备之间传文件
mise 与 uv	多语言版本和 Python 依赖分散
Loop、Rectangle、AeroSpace	原生窗口动作不足

14.2 第二步：回到官方来源

对每个候选检查：

- 官方安装路径
- 当前支持系统
- 权限要求
- 网络行为
- 商业许可
- 是否与现有工具重复
- 是否存在可解释的卸载路径

例如：

- OrbStack 的许可边界必须看官方 licensing 页面。
- Ghostty 推荐先零配置使用，因为官方文档明确这样设计。
- Stats 的 Sensors 和 Bluetooth 模块可能增加资源消耗，官方 README 给出了关闭建议。
- Maccy 支持 confidential 类型排除，但仍要理解剪贴板风险。

14.3 第三步：进入分层清单

不是所有好软件都进入默认层。一个候选只有在以下条件成立时才值得成为基线：

1. 大多数开发者都会遇到这个问题。
2. 工具职责清晰。
3. 来源稳定。
4. 权限和许可可控。
5. 卸载与迁移成本可接受。

其余工具放在按需层，遇到痛点再安装。

15. 三种工作站配置

不同角色不应该共享一份无限增长的安装清单。下面给出三套组合。

15.1 轻量通用开发

适合前端、后端、脚本和文档工作：

```
brew "bat"
brew "eza"
brew "fd"
brew "fzf"
brew "gh"
brew "git-delta"
brew "jq"
brew "mise"
brew "mole"
brew "ripgrep"
brew "uv"
brew "zoxide"
cask "ghostty"
```

特点：工具少、恢复快、终端体验完整。容器和启动器按需添加。

15.2 容器与云原生开发

在轻量通用开发基础上添加：

```
cask "orbstack"
```

再按项目需要使用 Docker Compose 或本地 Kubernetes。不要把 Kubernetes 当作所有项目默认入口。

特点：容器、Linux 和 Mac 文件系统边界清晰；需要额外关注许可、volume 和跨架构镜像。

15.3 效率增强桌面

在通用开发基础上，按痛点选择：

```
cask "raycast"
cask "jordanbaird-ice"
cask "stats"
cask "iina"
cask "keka"
cask "localsend"
```

窗口工具只选一个：

```
cask "loop"  
# 或 cask "rectangle"
```

特点：桌面动作更短，但权限、后台常驻和快捷键冲突需要定期复核。

16. 排障手册

排障时先确认层级，不要一上来重装系统或复制更多配置。

16.1 brew 找不到

```
which brew
echo "$PATH"
```

Apple Silicon 常见修复：

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```

确认后把正确语句写入 `.zshrc`。Intel Mac 路径不同。

16.2 项目工具版本错误

```
mise current
mise doctor
```

检查：

- 当前目录是否有 `mise.toml`
- 上级目录是否有覆盖配置
- 是否存在 `mise.lock`
- `shell` 是否已激活 `mise`
- 拉取的新配置是否已经审阅和信任

16.3 Python 包找不到

```
uv run python -V
uv sync
uv run python -c 'import sys; print(sys.executable)'
```

优先通过 `uv run` 执行项目命令，不要先回到全局 `pip install`。

16.4 Docker 异常

```
docker context show
docker compose ps
docker volume ls
```

检查：

- OrbStack 是否已启动
- context 是否为预期值
- 是否混用多个 Docker 实现
- bind mount 是否跨越了不合适的文件系统边界
- Apple Silicon 是否正在运行只提供 x86 的镜像

16.5 菜单栏工具不显示

Stats 官方 README 特别提醒，macOS 26 引入了菜单栏隐私控制。如果 Stats 已启动、模块已开启，但图标不显示，先查看 [系统设置 > 菜单栏](#)。

16.6 快捷键冲突

按顺序排查：

1. 系统键盘快捷键
2. 输入法切换
3. Spotlight
4. Raycast
5. 窗口管理器
6. 剪贴板工具
7. 编辑器快捷键

不要同时修改多个工具。一次只改一处，使用一天再判断。

16.7 磁盘空间紧张

```
mo analyze
```

先观察，再处理。大目录不等于垃圾目录。容器 volume、模拟器、SDK、模型文件和项目缓存都可能很大，但删除策略不同。

17. 最佳食用方式

这套工作流的价值，不在于第一天把系统打磨到极致，而在于每次变化都有入口。

第一天

- 完成第 2 章和第 3 章。
- 学会 `⌘ Space`、`⌘ Tab`、`⌘ ``、`Space`、`⇧ ⌘ G`、`⇧ ⌘ 5`。
- 保留最小 Brewfile 和 `.zshrc`。
- 不安装窗口管理器、剪贴板工具和菜单栏工具，除非痛点已经明确。

第一周

- 记录每天重复出现的摩擦。
- 只针对最高频痛点补一个工具。
- 对新权限做一次复核。
- 使用一个真实项目验证 `mise` 与 `uv`。
- 有容器需求时，用一个 `Compose` 项目验证 `OrbStack`。

第一个月

- 执行一次每月 SOP。
- 确认 `Time Machine` 真的有备份。
- 用 `brew bundle dump` 对比现状与契约。
- 用 `mo analyze` 理解磁盘分布。
- 把稳定的 dotfiles 纳入 `chezmoi` 或普通 `Git` 仓库。

长期

- 工具更新从“追新”变为“受控升级”。
- 软件推荐从“别人说好用”变为“它减少了哪一步”。
- 机器迁移从“手工回忆”变为“按契约恢复”。
- 系统优化从“批量执行”变为“备份、预演、审阅、再执行”。

| 18. 发布前自检

如果要把这套工作站配置交给团队，至少完成以下检查。

系统

- ☐ 系统版本已记录。
- ☐ FileVault 已开启，恢复方式已保存。
- ☐ Time Machine 已完成首次备份。
- ☐ 权限列表已复核。

安装契约

- ☐ Brewfile 已进入版本控制。
- ☐ `brew bundle check --file=./Brewfile` 通过。
- ☐ 没有未经审阅的 `tap` 或脚本。
- ☐ GUI 工具都有明确职责。

开发环境

- ☐ `.zshrc` 保持最小且可解释。
- ☐ `mise doctor` 通过。
- ☐ 项目包含 `mise.toml`，需要复现时包含 `mise.lock`。
- ☐ Python 项目使用 `pyproject.toml` 和 `uv.lock`。
- ☐ 容器项目有明确的 `Compose` 文件、`volume` 策略和架构说明。

维护

- ☐ 清理命令默认先 `--dry-run`。
- ☐ 没有无人值守深度清理任务。
- ☐ 大版本升级前有备份与恢复清单。
- ☐ 不再使用的权限、登录项和常驻模块已回收。

一台成熟的 Mac 工作站不需要显得复杂。它应该让日常路径更短，让故障边界更清楚，让下一台机器更容易恢复。

附录：macOS Developer Workstation Cheatsheet

这一章可以独立打印。第一次接入按顺序执行，日常使用按问题查动作和命令。

1. 新机 60 分钟

时间	动作	验收
0 - 10 分钟	系统更新、登录账户、开启 FileVault	系统设置 > 通用 > 软件更新 无待处理更新
10 - 20 分钟	连接 Time Machine, 确认恢复密钥保存位置	备份菜单能看到最近备份时间
20 - 30 分钟	安装 Command Line Tools 与 Homebrew	<code>xcode-select -p</code> 、 <code>brew --version</code>
30 - 40 分钟	应用基础 Brewfile	<code>brew bundle check --file=./Brewfile</code>
40 - 50 分钟	配置 Git、GitHub CLI、zsh、mise、uv	<code>gh auth status</code> 、 <code>mise doctor</code> 、 <code>uv --version</code>
50 - 60 分钟	按需安装 Ghostty、OrbStack, 完成冒烟测试	<code>docker run --rm hello-world</code>

```
xcode-select --install

# Homebrew 安装完成后, 根据安装器提示执行 shellenv。
eval "$(/opt/homebrew/bin/brew shellenv)"

brew bundle check --file=./Brewfile || brew bundle install --file=./Brewfile
gh auth login
```


2. 键位图例

符号	名称	常见语义
	Command	应用级主修饰键
	Option / Alt	替代动作、按词移动、显示细粒度选项
	Control	系统动作、终端动作、桌面切换
	Shift	反向、范围选择、组合动作
	Function / Globe	功能键、输入源和系统动作

3. 每天都值得记住的快捷键

快捷键	动作
 Space	Spotlight 搜索
 Tab	切换应用
 `	切换当前应用的窗口
 W	关闭窗口或标签
 Q	退出应用
 ,	打开当前应用设置
  Esc	强制退出应用
  Q	锁定屏幕
Space	Finder 中 Quick Look 预览
  3	截取全屏
  4	截取选区
  5	截图与录屏面板
 ↑	Mission Control
 ↓	当前应用的所有窗口

4. Finder

快捷键	动作
<code>⌘ ⌘ G</code>	前往文件夹
<code>⌘ ⌘ H</code>	打开用户目录
<code>⌘ ⌘ L</code>	打开下载目录
<code>⌘ ⌘ U</code>	打开实用工具目录
<code>⌘ ⌘ N</code>	新建文件夹
<code>⌘ I</code>	显示简介
<code>⌘ Delete</code>	移到废纸篓
<code>⌘ ⌘ V</code>	将剪贴板中的文件移动到当前位置
<code>⌘ ⌘ P</code>	显示或隐藏路径栏
<code>⌘ 1 / 2 / 3 / 4</code>	图标、列表、分栏、画廊视图
<code>⌘ ↑</code>	上级目录
<code>⌘ ↓</code>	打开所选项目
<code>⌘ [/ ⌘]</code>	后退 / 前进

5. 文本编辑

快捷键	动作
<code>⌘ ← / ⌘ →</code>	按词移动
<code>⌘ ← / ⌘ →</code>	行首 / 行尾
<code>⌘ ↑ / ⌘ ↓</code>	文首 / 文末
<code>⌘ Delete</code>	删除左侧一个词
<code>fn Delete</code>	向右删除
<code>⌘ 加移动键</code>	扩展选择范围
<code>⌘ Z</code>	撤销
<code>⌘ ⌘ Z</code>	重做

6. Homebrew 与 Brewfile

```
# 日常查询
brew search <name>
brew info <name>
brew list
brew outdated

# 应用契约
brew bundle check --file=./Brewfile
brew bundle install --file=./Brewfile

# 维护
brew update
brew upgrade
brew cleanup
brew doctor

# 导出现状, 必须审阅 diff
brew bundle dump --global --force --describe

# 危险: 只看差异, 不要直接加 --force
brew bundle cleanup --file=./Brewfile
```

Apple Silicon 默认 Homebrew 前缀为 `/opt/homebrew` , Intel Mac 默认前缀为 `/usr/local` 。

7. 推荐基础 Brewfile

```
brew "bat"
brew "eza"
brew "fd"
brew "fzf"
brew "gh"
brew "git-delta"
brew "jq"
brew "mise"
brew "mole"
brew "ripgrep"
brew "uv"
brew "zoxide"

cask "ghostty"
# cask "orbstack" # 有容器或 Linux 需求时启用
# cask "raycast" # 原生 Spotlight 不够用时启用
```

8. zsh 最小配置

```
# ~/.zshrc
if [[ -x /opt/homebrew/bin/brew ]]; then
    eval "$(/opt/homebrew/bin/brew shellenv)"
fi

eval "$(mise activate zsh)"
eval "$(zoxide init zsh)"

alias ll='eza -lah --group-directories-first'
alias cat='bat --paging=never'
```

9. Git 与 GitHub CLI

```
git config --global --edit
git config --global init.defaultBranch main
git config --global pull.ff only
git config --global fetch.prune true

gh auth login
gh auth status
ssh -T git@github.com
```

10. mise

```
# 交互 shell 已在 ~/.zshrc 中激活
mise doctor
mise ls
mise current

# 全局默认, 只保留少量通用项
mise use --global node@lts

# 项目内写入 mise.toml
mise use node@lts
mise use python@3.12
mise install
mise run test

# 团队可复现
mise settings lockfile=true
mise lock
git add mise.toml mise.lock
```

拉取陌生项目后, 先审阅 `mise.toml`、任务和 `hooks`, 再执行信任操作。

11. uv

```
brew install uv

uv init hello-world
cd hello-world
uv add requests
uv run python -c 'import requests; print(requests.__version__)'

uv lock
uv sync
uv tool install ruff
```

Python 项目依赖写入 `pyproject.toml` 与 `uv.lock`。不要把项目依赖堆到全局 Python。

12. OrbStack

```
brew install orbstack

docker context show
docker run --rm hello-world
docker compose up -d
docker compose ps
docker compose down

orb create ubuntu devbox
orb -m devbox uname -a
orb logs devbox
```

场景	做法
容器持久数据	优先 volume; 需要从 Mac 观察时使用 <code>~/OrbStack/docker</code>
Apple Silicon 运行 x86 镜像	显式使用 <code>--platform linux/amd64</code> , 只在确有需要时启用
Linux 中访问 Mac 文件	使用 <code>/mnt/mac</code>
Mac 中访问 Linux 文件	使用 <code>~/OrbStack</code> 或 Finder 集成
商业使用	先阅读官方 licensing 页面
Docker TCP	不开放未认证的 <code>0.0.0.0:2375</code>

13. Mole

```
brew install mole

# 默认从只读动作开始
mo status
mo analyze

# 删除动作先预演
mo clean --dry-run
mo uninstall --dry-run
mo purge --dry-run

# 回看日志
mo history
mo history --json
```

不要把深度清理写进定时任务。删除前先备份，阅读列表，确认 VPN、开发缓存、模拟器和挂载卷等工作资产不会受影响。

14. 按痛点安装

痛点	先试原生	第三方候选
启动、动作、Quicklinks、Snippets	Spotlight	Raycast
窗口排列	macOS 原生平铺	Loop / Rectangle / AeroSpace 三选一
菜单栏拥挤	系统设置精简	Ice
剪贴板历史	无	Raycast Clipboard 或 Maccy 二选一
系统状态	活动监视器	Stats
媒体播放	QuickTime Player	IINA
压缩解压	Finder	Keka
跨平台局域网传输	AirDrop	LocalSend
外接显示器	系统显示器设置	BetterDisplay
GUI 卸载	手工移除	Pearcleaner
GUI 安装 Homebrew cask	Brewfile	Applite

15. 日常 SOP

每天

```
git status --short --branch  
mise current  
docker compose ps
```

每周

```
brew update  
brew outdated  
brew bundle check --file=./Brewfile  
mo status
```

每月

```
brew upgrade  
brew cleanup  
brew doctor  
mo analyze
```

同时检查 Time Machine 最近备份、权限列表、登录项、磁盘余量和不再使用的菜单栏模块。

大版本升级前

1. 完成一次 Time Machine 备份。
2. 导出并审阅 Brewfile。
3. 提交 dotfiles 与项目变更。
4. 记录关键应用许可、VPN、证书和组织策略。
5. 暂停高风险清理与批量修改。

16. 排障顺序

现象	第一检查	第二检查
<code>brew</code> 找不到	<code>which brew</code>	<code>eval "\$(/opt/homebrew/bin/brew shellenv)"</code>
项目工具版本错	<code>mise current</code>	审阅 <code>mise.toml</code> 与 <code>mise.lock</code>
Python 包找不到	<code>uv run python -V</code>	<code>uv sync</code>
Docker 命令异常	<code>docker context show</code>	打开 OrbStack, 检查容器与 volume
菜单栏监控不显示	模块是否开启	系统设置 > 菜单栏 权限
快捷键失效	系统快捷键冲突	启动器、输入法和窗口工具冲突
磁盘空间紧张	<code>mo analyze</code>	审阅大目录, 再决定是否删除
shell 变慢	<code>zsh -xlic exit</code>	暂时移除插件, 保留最小配置

附录：资料来源与边界

事实优先来自 Apple 支持文档和各项目官方文档；社区讨论用于发现真实痛点、候选工具和反例。

Method

本手册在 2026-06-02 完成资料收集与核验。事实优先来自 Apple 支持文档和各项目官方文档；Homebrew API 用于确认 Brewfile 标识；Linux.do 公开讨论用于发现候选工具、使用场景和反例。社区内容不作为安全边界或高风险命令的唯一依据。

逐条链接保存在 `data/source-map.tsv`。以下为正文使用频率最高的来源。

Apple

- Find out which macOS your Mac is using: 系统版本口径。Apple 在 2026-05-11 发布的页面中列出 macOS Tahoe 26.5。
- Mac keyboard shortcuts: 修饰键、应用切换、Finder、截图、锁屏和文本编辑快捷键。
- Take actions and shortcuts in Spotlight: Spotlight 能力。
- Trackpad gestures, three finger drag 与 Hot Corners: 触控板和桌面动作。
- Window tiling settings: 原生窗口平铺设置。
- FileVault 与 Time Machine: 磁盘加密和备份。
- Privacy & Security、Accessibility 与 screen recording: 权限治理。

Workstation Contract

- Homebrew Installation: 默认前缀、Command Line Tools 和支持系统。
- Brew Bundle and Brewfile: `brew bundle check`、`dump`、`cleanup` 与版本控制工作流。
- Homebrew Analytics: 匿名统计说明和关闭命令。
- Ghostty binary install、configuration 与 shell integration: 安装、零配置理念和配置文件路径。
- mise getting started、configuration、tasks 与 lockfile: 运行时、任务与锁文件。
- uv installation、projects 与 tools: Python 项目工作流。
- GitHub CLI auth login: GitHub CLI 登录。
- chezmoi install 与 quick start: dotfiles 管理。

Containers and Maintenance

- OrbStack quick start、Docker、Linux machines、Kubernetes、efficiency 与 licensing: 容器、Linux、资源行为、安全和商业许可。
- Mole README 与 security policy: 清理命令、`--dry-run`、操作日志和保护边界。

Optional Tools

- Raycast window management、clipboard history、Quicklinks、Snippets 与 security and privacy。
- Ice、Stats、IINA、Keka、LocalSend、Maccy。
- BetterDisplay、Rectangle、Loop、AeroSpace。

- [Pearcleaner](#)、[Applite](#)、[mas](#)。

Linux.do Community Reading

- [macOS 软件与配置讨论](#)
- [沉淀数月使用，我推荐的 Mac 软件](#)
- [Mac 软件推荐分享](#)
- [Mac 效率与软件讨论](#)
- [开发环境管理迁移讨论](#)
- [新机一键安装讨论](#)
- [第一台 MacBook 软件讨论](#)
- [Mac App 推荐讨论](#)

Boundaries

- 本手册不是 Apple、Homebrew 或任何第三方项目的官方出版物。
- 推荐项不等于永久安装项。团队许可、组织 MDM、合规要求和实际工作负载优先。
- 软件界面、价格、许可和系统菜单会变化。安装前应再次阅读对应官方页面。
- 文中示例不会替代备份、权限复核和代码审阅。